



دانشگاه تهران
پردیس دانشکدگان فنی
دانشکده برق و کامپیوتر



سیگنال و سیستم

تمرین کامپیوتری شماره ۴

استاد: دکتر سعید اخوان

نام و نام خانوادگی

سید علی رضاعی

شماره دانشجویی

۸۱۰۱۰۳۴۳۲

بهار ۱۴۰۵

بخش اول:

تمرین ۱-۱)

```
my_mapset.m  Part1_check_before_functionized.m  coding.m
1 - mapset = cell(2,32);
2
3 - characters = 'abcdefghijklmnopqrstuvwxyz ,!";';
4
5 - for i = 1:32
6 -     mapset{1,i} = characters(i);
7 -     mapset{2,i} = dec2bin(i-1,5);
8 - end
9
10 - save('my_mapset.mat', 'mapset');
11
```

توضیحات کد:

برای درست کردن مپ ست ابتدا یه سلول ۲ در ۳۲ درست کردم و بعد میاییم اون کاراکتر هایی که به عنوان مپ ست میخوایم رو توی متغیر characters ذخیره میکنیم. سپس با استفاده از یک حلقه ی فور ان را مقدار دهی میکنیم که متناظر هر کدام از این ها چه عدد باینری ای است برای منظور از تابع dec2bin استفاده میکنیم و در نهایت نیز اون سلول رو به اسم my_mapset سیو میکنیم.

تمرین ۱-۲)

```
1 - function secretImage = coding(message, myGrayImage, mapset)
2
3 -     if message(end) ~= ';'
4 -         message = [message, ';'];
5 -     end
6
7
8 -     message2binary = [];
9
10 -    for i = 1:length(message)
11 -        found_character = find(strcmp(cellstr(mapset(1,:)), message(i)));
12 -        message2binary = [message2binary, mapset(2, found_character)];
13 -    end
14
15
16
17 -    [rows, cols] = size(myGrayImage);
18 -    block_size = 10;
19
20 -    max_blockVar = -1;
21 -    idx_best_block = [1, 1];
22
23
24 -    for n = 1:block_size:(rows-block_size+1)
25 -        for m = 1:block_size:(cols-block_size+1)
26
27 -            block = double(myGrayImage(n:n+block_size-1, m:m+block_size-1));
28 -            blockVar = var(block(:));
29
30 -            if blockVar > max_blockVar
31 -                max_blockVar = blockVar;
32 -                idx_best_block = [m, n];
33 -            end
34
35 -        end
36 -    end
37
```

```

38 -
39 -     if length(message2binary) > block_size^2
40 -         error('length of the message should be shorter')
41 -     end
42 -
43 -     best_col = idx_best_block(1);
44 -     best_row = idx_best_block(2);
45 -
46 -     secretImage = myGrayImage;
47 -
48 -
49 -     bit_counter = 1;
50 -     for r_offset = 0:(block_size - 1)
51 -         for c_offset = 0:(block_size - 1)
52 -             r = best_row + r_offset;
53 -             c = best_col + c_offset;
54 -             if bit_counter <= length(message2binary)
55 -                 bit_val = str2double(message2binary(bit_counter));
56 -                 secretImage(r, c) = bitset(secretImage(r, c), 1, bit_val);
57 -                 bit_counter = bit_counter + 1;
58 -             end
59 -         end
60 -     end
61 -
62 -
63 -     figure;
64 -     subplot(1,2,1); imshow(myGrayImage); title('Original Image');
65 -     subplot(1,2,2); imshow(secretImage); title('Image with Hidden Message');
66 - end
67 -

```

توضیح کد:

به طور کلی این کد را میتوان به ۳ بخش تقسیم بندی کرد

بخش اول: تبدیل مسیج به باینری

در این بخش از خط ۳ شروع میشود ابتدا بررسی میکنیم که آیا مسیج ما علامت نقطه ویرگول در انتهای آن دارد یا خیر، در صورتی نداشتن آن به آخر آن اضافه میکنیم در ادامه یک متغیر خالی تعریف میکنیم که میخواهیم مسیج باینری شده را در داخل آن ذخیره سازی بکنیم در داخل حلقه فور از خط ۱۰ ، از یک تا طول پیام اون ابتدا ردیف اول مپ ست و تمام عناصر ردیف اول را به رشته تبدیل میکنیم با استفاده از تابع cellstr و سپس این رشته ها را با مسیج i ام مقایسه میکنیم با استفاده از تابع strcmp ، خروجی این تابع به ازای مقادیری که با هم مساوی بوده ۱ و مقادیری که با هم مساوی نبوده صفر است سپس از تابع فایند استفاده میکنم تا ایندکس اون جایی که یک شده رو بهمون بده و آن را داخل مغییر finded_character ذخیره سازی میکنیم. و سپس میگییم که اون متغییر خالی ما رو که اسمشو گذاشتیم message2binary رو پس کن با استفاده از اون مپ ست که ردیف ۲ اش که باینری هست و اون ایندکسی که پیداش کردیم اینو اضافه کن به اون message2binary

بخش دوم: پیدا کردن بهترین قسمت عکس برای پنهان کردن دیتا

در خط ۱۷ ابتدا با استفاده از تابع imread عکس را میخوانیم و سپس اندازه آن را در rows , cols ذخیره میکنیم در خط ۱۸ اندازه بلوک هایی که میخوانیم توش دیتا ذخیره کنیم رو مشخص میکنیم من اینجا اومدم و ۱۰ انتخاب کردم یعنی مسیج ها توی یه سری بلوک های ۱۰ در ۱۰ ذخیره میشه و در دوتا مغییر تعریف میکنیم که بعدا قراره عوض کنیم حالا یکیش بیشترین واریانس بلوکه و یکیشم مختصاتشه

یه حلقه فور در هم داریم که می‌گیم که بیا از ۱ تا $row - block_size + 1$ بلوک سایز بلوک سایز تا برو و توی حلقه فور زیری هم برای ستون هاش چیکار کن؟ بیا این (n تا $n + block_size + 1$ و از m تا $m + block_size + 1$) قسمت از عکس طوسی رو انتخابش بکن و تبدیل اش بکن به دابل بعد اینو بریز توی block این میشه اون بلوکمون حالا بیا واریانس اش رو بگیر و بریز توی blockVar. $var(block(:));$ یعنی اینکه بیا همه عنصر های داخل بلوک رو بین اش واریانس بگیر بعد میاییم با دستور خط ۳۰ تا ۳۳ که خیلی تیپیکاله توی برنامه نویسی ماکسیموم اش رو پیدا میکنیم و بعد توی idx_best_block اون لوکیشن ام و ان اش رو ذخیره میکنیم.

بخش سوم: ذخیره دیتا در داخل بلوک انتخاب شده

در خط ۳۹ تا ۴۱ بررسی میکنیم که آیا این بلوک قابلیت ذخیره این پیام رو داره یا نه از لحاظ اندازه منظورم هست

و اگر نباشه ارور بده

در ادامه میاییم در بخش حلقه فور را بررسی میکنیم

در این بخش از برنامه، فرآیند جاسازی پیام باینری درون تصویر انجام می‌شود. ابتدا یک شمارنده به نام bit_counter با مقدار اولیه ۱ تعریف می‌شود تا مشخص کند که در هر لحظه، کدام بیت از رشته message2binary باید در تصویر مخفی شود. این شمارنده نقش یک اشاره‌گر را ایفا می‌کند که با جاسازی موفق هر بیت، یک واحد افزایش می‌یابد.

سپس برای پیمایش تمام ۱۰۰ پیکسل بلوک منتخب از دو حلقه تو در تو استفاده می‌شود. حلقه بیرونی متغیر r_offset را از صفر تا ۹ تغییر می‌دهد که نشان‌دهنده انحراف ردیف از نقطه شروع بلوک best_row است. به طور مشابه، حلقه درونی نیز با متغیر c_offset از صفر تا ۹، انحراف ستون از نقطه شروع بلوک best_col را مشخص می‌کند. از آنجا که block_size اکنون برابر با ۱۰ است، عبارت $block_size - 1$ دقیقاً همان ۹ می‌شود و ترکیب این دو متغیر امکان دسترسی به تک‌تک پیکسل‌های بلوک 10×10 را به صورت سطری (Row-Major) فراهم می‌کند.

در داخل حلقه داخلی، مختصات دقیق هر پیکسل با فرمول $r = best_row + r_offset$ و $c = best_col + c_offset$ محاسبه می‌شود. در ادامه یک شرط کنترلی بررسی می‌کند که آیا شمارنده bit_counter از طول کل رشته باینری پیام بزرگتر نشده باشد. این شرط برای مواقعی ضروری است که طول پیام کمتر از ۱۰۰ بیت (ظرفیت کامل یک بلوک 10×10) باشد، تا برنامه تلاش نکند بیتی ناموجود را پردازش کند و عملیات جاسازی تنها به اندازه طول واقعی پیام انجام شود.

هنگامی که شرط برقرار باشد، ابتدا بیت جاری از رشته message2binary که به صورت کاراکتری ('۰' یا '۱') ذخیره شده، توسط تابع str2double به مقدار عددی صفر یا یک تبدیل می‌شود. سپس عملیات اصلی پنهان‌سازی با استفاده از تابع bitset صورت می‌پذیرد. این تابع مقدار پیکسل واقع در مختصات (r, c) را دریافت کرده و کم‌ارزش‌ترین بیت آن (بیت شماره ۱) را با bit_val جایگزین می‌کند، بی‌آنکه سایر هفت بیت پیکسل تغییری کنند. نتیجه نهایی که همان پیکسل حامل یک بیت از پیام است، مجدداً در ماتریس secretImage در همان مختصات ذخیره می‌شود. در انتهای هر تکرار موفق، شمارنده bit_counter یک واحد افزایش می‌یابد تا به بیت بعدی پیام اشاره کند. با اتمام دو حلقه و پیمایش کامل بلوک 10×10 ، پیام به طور کامل در پیکسل‌های بلوک منتخب جاسازی شده است و پیکسل‌های مازاد بلوک (در صورتی که پیام کوتاه‌تر از ۱۰۰ بیت باشد) بدون تغییر باقی می‌مانند.

در انتها نیز دو تصویر را پلات میکنیم.

تمرین ۱-۳)

```
my_mapset.m  Part1_check_before_functionized.m  coding.m  p1_main.m
1 - load my_mapset.mat;
2 - myImage = imread('testPic.jpg');
3 - grayImg = rgb2gray(myImage);
4
5
6 - output_image = coding('signal', grayImg, mapset);
7
```

در این قسمت عکس و مپ ست را میخوانیم و لود میکنیم و به طوسی تبدیل کرده و سپس تابع را فراخوانده و مقادیر را به آن میدهیم. نتیجه آن به صورت زیر است:

Original Image



Image with Hidden Message



همان طور که مشاهده میشود تفاوتی به چشم دیده نمیشود و تغییراتی در تصویر اصلاً مشخص نیست زیرا ما LSB آن را تغییر دادیم و این یعنی به تفاوت خیلی خیلی جزئی در آن که چشم انسان نمیتواند آن را دیتکت کند.

تمرین ۱-۴)

ابتدا به تغییر کوچیک توی کد قسمت قبل دادم برای محاسبه ی واریانس اومدم بیت آخری رو صفر کردم و واریانس همه رو گرفتم که موقعی که میخوام دیکد بکنم به مشکل برخوردم همه با به استراتژی اون قسمت واریانس اش در بیاد

```
block = double(myGrayImage(n:n+block_size-1, m:m+block_size-1));
block_stable = bitset(block, 1, 0);
blockVar = var(block_stable(:));
```

یه اتفاق خیلی باحال افتاد من اومدم و فایل هامون با فرمت jpg ذخیره کردم این فرمت اینطوریه که میاد و فشرده سازی انجام میده و LSB هارو تغییر میده من کلی باگ خورده بودم که چرا دارم اشتباه دیکد میکنم در صورتی که کدم که کامل درسته! بعد فهمیدم بخاطر این فرمتشه و فرمتی که اصلاً بیت هارو تغییر نمیده PNG هست.

در ادامه به بررسی کد و نتایج میپردازیم.

```

1  function decoded_message = decoding(myGrayImage, mapset, block_size)
2  [rows, cols] = size(myGrayImage);
3
4  max_blockVar = -1;
5  idx_best_block = [1, 1];
6
7
8  for n = 1:block_size:(rows-block_size+1)
9      for m = 1:block_size:(cols-block_size+1)
10
11         block = myGrayImage(n:n+block_size-1, m:m+block_size-1);
12         block_stable = bitset(block, 1, 0);
13         blockVar = var(double(block_stable(:)));
14
15         if blockVar > max_blockVar
16             max_blockVar = blockVar;
17             idx_best_block = [m, n];
18         end
19     end
20 end
21
22 best_col = idx_best_block(1);
23 best_row = idx_best_block(2);
24
25
26 extracted_bits = '';
27 for r_offset = 0:(block_size - 1)
28     for c_offset = 0:(block_size - 1)
29         r = best_row + r_offset;
30         c = best_col + c_offset;
31
32         bit_val = bitget(myGrayImage(r, c), 1);
33         extracted_bits = [extracted_bits, num2str(bit_val)];
34     end
35 end
36
37
38 decoded_message = '';
39 current_bits = '';
40
41 for i = 1:length(extracted_bits)
42     current_bits = [current_bits, extracted_bits(i)];
43
44     idx = find(strcmp(mapset(2,:), current_bits));
45
46     if ~isempty(idx)
47         char_found = mapset(1, idx(1));
48         decoded_message = [decoded_message, char_found];
49         current_bits = '';
50
51         if char_found == ';'
52             break;
53         end
54     end
55 end
56
57
58
59 figure;
60 imshow(myGrayImage);
61 text(best_col, max(1, best_row - 15), decoded_message, 'Color', 'red', 'FontSize', 14, 'FontWeight', 'bold');
62 end

```

```

1  % --- Main Script ---
2  clc; clear; close all;
3
4
5  load('my_mapset.mat');
6  myGrayImage = imread('stego_image.png');
7  block_size = 10;
8
9
10 decoded_message = decoding(myGrayImage, mapset, block_size);
11
12 disp(['Final Decoded message is: ', decoded_message]);
13

```

در این تابع خیلی از قسمت هاش مشابه قسمت قبل است لذا از آن قسمت ها سریع عبور میکنیم در بخش اول که مربوط به دیتکت کردن قسمت مناسب برای وارانس است دقیقا مشابه قسمت قبلی است و همه ی LSB های آن صفر میکنیم که مشابه هم وارانس ها به دست بیاد دقیقا و سپس دقیقا برعکس اینکه بیت هارو ست کنیم bitget استفاده میکنیم با همون استراتژی دقیق و توی extracted_bits ذخیره میکنیم در ادامه میاییم با توجه به مپ ست اینا رو به حروف تبدیل میکنیم و میگیریم اگه رسیدی به ؛ ، Break بکن و بیا بیرون در ادامه آن را روی شکل هم نشان دادیم در محل حدودی نتایج به شکل زیر است.

```

Final Decoded message is: signal;
>>

```



تمرین ۱-۵)

همان طور گفته شد یکی از باگ های این روش همین است که توی ارسال پیام یه نویز خیلی کوچیک روی تصویر سوار بشه چون داریم روی LSB تغییرات ایجاد میکنیم خیلی تحت تاثیر نویز و تغییرات کوچیک قرار میگیره، حتی اون گیر فرمت و اینا رو هم داره یعنی باید دقیق دقیق همون انتقال پیدا کنه بدون هیچ کم و کسری. در روش های جدی تر از روش های آماری استفاده میکنند و کار های خیلی پیچیده تری انجام میشود که این مشکل هارو برطرف بکنند.

بخش دوم:

کد بخش دوم

```
1 - clc; clear; close all;
2
3 - % --- 1. Load Video ---
4 - videoFile = '2-3 VariableSize.mp4';
5 - if ~exist(videoFile, 'file')
6 -     error('Video file not found. Please check the file path.');
```

7 - end

8 - vid = VideoReader(videoFile);

9

10 - % --- Output Video Settings ---

11 - outputFile = 'tracked_airplane_3.mp4';

12 - outputVideo = VideoWriter(outputFile, 'MPEG-4');

13 - outputVideo.FrameRate = vid.FrameRate;

14 - open(outputVideo);

15

16 - % --- 2. Read first frame and Select object ---

17 - firstFrame = readFrame(vid);

18 - firstGray = rgb2gray(firstFrame);

19

20 - figure('Name', 'Select Object');

21 - imshow(firstFrame);

22 - title('Draw a rectangle around the object and double-click.');

23 - h = imrect;

24 - position = wait(h);

25 - close;

26

27 - trackBox = position; % Current tracked position [x, y, width, height]

28 - frame1_gray = firstGray;

29

30 - % --- 3. Process Frames ---

31 - hFig = figure('Name', 'Tracked Video');

32

33 - while hasFrame(vid)

34 - frame2 = readFrame(vid);

35 - frame2_gray = rgb2gray(frame2);

36

37 - % --- Step 1 & 2: Difference and Thresholding ---

38 - diffFrame = imabsdiff(frame2_gray, frame1_gray);

39 - bw = diffFrame > 20; % Sensitivity threshold

40

41 - % --- Step 3: Morphology ---

42 - bw = bwareaopen(bw, 15);

43 - se = strel('disk', 3);

44 - bw = imclose(bw, se);

45

46 - % --- Step 4 & 5: Detection and Matching ---

47 - stats = regionprops(bw, 'BoundingBox', 'Area', 'Centroid');

48

49 - bestMatch = [];

50 - minDist = inf;

51

52 - % Define allowed movement (in pixels) - e.g., plane can't jump 100 pixels in 1 frame

53 - maxMovement = 50;

54

55 - % Define size constraints (optional: plane size won't change drastically)

56 - % You can tune these values based on your target size

57 - minArea = 20;

58 - maxArea = 500;

59

60 - if ~isempty(trackBox) && numel(trackBox) >= 4

61 - prevCenter = [trackBox(1) + trackBox(3)/2, trackBox(2) + trackBox(4)/2];

62

63 - for k = 1:length(stats)

64 - currCenter = stats(k).Centroid;

65 - dist = norm(currCenter - prevCenter);

66 - currArea = stats(k).Area;

67

68 - % NEW: Add strict filtering

69 - % 1. Distance constraint (must be close to previous)

70 - % 2. Area constraint (must be within size limits)

71 - if dist < minDist && dist < maxMovement && ...

72 - currArea > minArea && currArea < maxArea

73

74 - minDist = dist;

75 - bestMatch = stats(k).BoundingBox;

76 - end

77 - end

78 - end

79

80


```

81 % --- Step 6: Update and Draw ---
82 frameOut = frame2;
83 if ~isempty(bestMatch)
84     trackBox = bestMatch; % Update successful
85     frameOut = insertShape(frameOut, 'Rectangle', trackBox, 'Color', 'green', 'LineWidth', 3);
86 elseif ~isempty(trackBox)
87     % If not found, use the last known position (Yellow box)
88     frameOut = insertShape(frameOut, 'Rectangle', trackBox, 'Color', 'yellow', 'LineWidth', 3);
89 end
90
91 % --- Step 7: Display and Write ---
92 if ishandle(hFig)
93     imshow(frameOut);
94     title('Tracking (Green: Found, Yellow: Estimated)');
95     drawnow;
96 end
97
98 writeVideo(outputVideo, frameOut);
99 frame1_gray = frame2_gray;
100 end
101
102 close(outputVideo);
103 disp('Processing completed. Output video saved.');
```

توضیحات مربوط به کد:

همانطور که مشاهده میشود چون کد طولانی است آن را بخش بخش کردم

بخش اول تنظیمات اولیه

در ابتدا تعیین نام فایل ویدیویی؛ `videoFile = '2-3_VariableSize.mp4'`؛ در ادامه بررسی میکنیم که یا فایل در مسیر وجود دارد یا خیر این کار برای این است که از توقف برنامه با خطای ناموجود بودن فایل جلوگیری کنیم در ادامه ویدیو مربوطه را میخوانیم.

دستور `videowriter(outputFile, 'MPEG-4')` این دستور فایل خروجی را با فرمت استاندارد MP4 فشرده سازی میکند و دیگه فایل حجیم نمیشه مثلاً AVI ولی در عین حال کیفیت مناسبی داره.

در ادامه خط ۱۳ فریم ریت ویدیو خروجی را با فریم ریت ویدیو ورودی یکسان میکنیم این کار برای این است که مثلاً بهو خود متلب با پیش فرض های خودش نیاد و به مقدار دلخواه بذاره و بعد حس کند شدن ویدیو یا تند شدن ویدیو به آدم بده برای اینکه همونی که هست بمونه.

بخش دوم انتخاب هدف

`readFrame(vid)` اولین فریم ویدیو را برای انتخاب توسط کاربر فراخوانی میکند. و سپس تصویر به به خاکستری تبدیل میکنیم

تابع `imrect` اجازه میدهد به کاربر که به مستطیل بکشه و خروجی `[x, y, width, height]` رو بهمون میده این رو میریزیم توی `h` و بعد میگیریم بهش که صبر کنه تا زمانی که کاربر در داخل مستطیل دبل کلیک نکرده کاری نکن بعدش خروجی را داخل پوزیشن ذخیره سازی میکنیم و در نهایت این فریم را چون هنوز وارد حلقه اصلی نشدیم اینو به عنوان اولین فریم قبلی ذخیره میکنیم در فریم دوم، برنامه سراغ این متغیر میآید تا ببیند نسبت به فریم اول چه تغییری ایجاد شده است.

بخش سوم حلقه پردازش ویدیو

این بخش از کد، موتور پردازشی و هسته اصلی سیستم ردیابی (Tracking Loop) را تشکیل می‌دهد. منطق کلی این حلقه بر پایه «مقایسه فریم به فریم» استوار است؛ به این صورت که در هر تکرار، تصویر فعلی با تصویر پیشین تفاضل‌گیری شده و تغییرات (حرکت) شناسایی می‌شوند. سپس با استفاده از فیلترهای هندسی و مکانی، از میان تمامی حرکت‌های شناسایی‌شده در تصویر، تنها حرکتی که با مشخصات (موقعیت و ابعاد) هدف ما مطابقت دارد، به عنوان هوایما انتخاب و دنبال می‌شود.

در بخش آشکارسازی حرکت، کد از تابع `imabsdiff` استفاده می‌کند تا اختلاف مطلق سطح خاکستری پیکسل‌ها را بین فریم فعلی (`frame2_gray`) و فریم قبلی (`frame1_gray`) محاسبه کند. خروجی این تابع تصویری است که در آن فقط نقاطی که تغییر وضعیت داده‌اند (حرکت کرده‌اند) مقادیر غیرصفر دارند. بلافاصله پس از آن، یک عملیات آستانه‌گذاری (`bw = diffFrame > 20`) انجام می‌شود که پیکسل‌های با تغییر کم (نویز) را حذف و پیکسل‌های با تغییر زیاد را به صورت سفید مطلق نمایش می‌دهد تا یک تصویر باینری (سیاه و سفید) جهت پردازش‌های بعدی آماده شود. در مرحله مورفولوژی، کد به دنبال حذف نویزهای ناخواسته و یکپارچه‌سازی هدف است. تابع `bwareaopen` اشیاء بسیار کوچک (کمتر از ۱۵ پیکسل) را که معمولاً نویزهای الکترونیکی سنسورهای IR هستند، حذف می‌کند. سپس، از یک المان ساختاردهنده دایره‌ای با شعاع ۳ پیکسل (`strel(disk, 3)`) در ترکیب با تابع `imclose` استفاده می‌شود تا شکاف‌های کوچک درون جسم متحرک پر شده و مرزهای آن برای شناسایی یکپارچه، هموارتر شوند؛ این کار باعث می‌شود هواپیما در خروجی باینری به جای چند نقطه پراکنده، به صورت یک تکه و متصل دیده شود. بخش شناسایی و تطبیق (`Detection & Matching`)، هوشمندانه‌ترین قسمت کد است که در آن از `regionprops` برای استخراج ویژگی‌های هندسی تمام مناطق سفید (حرکت‌ها) استفاده می‌شود. در این قسمت، ابتدا مرکز (`Centroid`) آخرین موقعیت شناخته‌شده‌ی هواپیما در `trackBox` محاسبه می‌شود. سپس برنامه در یک حلقه‌ی `for`، تمام اشیاء شناسایی‌شده در فریم فعلی را بررسی می‌کند. در اینجا دو فیلتر سخت‌گیرانه اعمال می‌شود: اولاً فاصله اقلیدسی (`norm`) بین مرکز شیء جدید و موقعیت قبلی نباید از `maxMovement` بیشتر باشد (تا از پرش‌های ناگهانی ردیاب جلوگیری شود) و ثانیاً مساحت شیء (`Area`) باید در محدوده منطقی `minArea` و `maxArea` باشد تا از انتخاب اشیاء بسیار کوچک یا بسیار بزرگ (مثل ابرها) جلوگیری شود. در صورتی که شیئی شرایط این فیلترها را داشته باشد، مختصات آن به عنوان `bestMatch` جایگزین می‌شود. این ساختار نشان می‌دهد که چرا برنامه در برابر نویز و اشیاء مزاحم مقاوم است؛ زیرا تنها به تغییرات "کوچک" در فضای "اطراف" موقعیت قبلی هدف اعتماد می‌کند. این مکانیسم تطبیق، باعث می‌شود ردیاب هویت هدف خود را در میان سایر اشیاء متحرک موجود در ویدیو، در طول زمان حفظ کند.

نتایج کلی در ویدیوهای ذخیره شده در فایل قابل مشاهده هستند در گزارش کار صرفاً چند نمونه از تصاویر را قرار خواهیم داد.

